

# 1. Guion Técnico de Apoyo para Exposición (Audiencia Experta)

## 1.1. Arquitectura General

- Arquitectura desacoplada tipo cliente-API.
- Separación estricta entre:
  - Capa de presentación (HTML/CSS/JS).
  - Capa de servicios (API REST).
- API implementada en FastAPI:
  - Tipado fuerte.
  - Serialización automática.
  - OpenAPI nativo.
- Persistencia en PostgreSQL mediante SQLAlchemy ORM.

## 1.2. Capa de Datos

### 1.2.1. Configuración (`database.py`)

- Engine con pool de conexiones.
- `SessionLocal` como unidad transaccional.
- Inyección de sesión en endpoints.

### 1.2.2. Modelo (`models.py`)

- Entidad `User` mapeada con SQLAlchemy.
- Restricción de unicidad:
  - `correo` con `unique=True`.
- Contraseña:
  - Nunca se persiste en claro.
  - Solo hash persistente.

## 1.3. Capa de Esquemas

### 1.3.1. Validación (`schemas.py`)

- Uso de Pydantic como frontera semántica.
- DTOs:
  - `UserCreate`.
  - `UserUpdate`.
- Validación previa a:
  - Lógica de negocio.
  - Acceso a datos.

## 1.4. Capa de Seguridad

### 1.4.1. Criptografía (`security.py`)

- Hashing con `bcrypt` (`passlib`).
- Tokens JWT:
  - Payload: identidad + rol.
  - Expiración definida.
- Autorización vía dependencia:
  - `verify_token`.

## 1.5. Lógica de Negocio

### 1.5.1. CRUD (`crud.py`)

- Acceso directo a ORM.
- Función crítica:
  - `actualizar_usuario`.
- Uso de:
  - `exclude_unset=True`.
- Implica:
  - Actualización parcial.

- No sobreescritura involuntaria de credenciales.

### 1.5.2. Controlador (`main.py`)

- Orquestación de dependencias.
- Implementación de RBAC.
- Roles privilegiados:
  - director.
  - coordinador.
- Restricción mediante:
  - Evaluación del rol desde JWT.
  - Excepción HTTP 403.

## 1.6. Frontend

### 1.6.1. Estilo (`style.css`)

- Neomorfismo:
  - Doble `box-shadow`.
  - Ilusión de relieve.

### 1.6.2. Cliente (`login.js`, `dashboard.js`)

- Consumo REST vía Fetch API.
- Persistencia de sesión:
  - Token en `localStorage`.
- Filtro local:
  - `filtrarTabla()`.
  - Evita round-trips innecesarios.
- Control visual de permisos:
  - UI reactiva al rol.
  - Seguridad real delegada al backend.

## 1.7. Cierre Técnico

- Pipeline completo:
  - Validación cliente  $\rightarrow$  JWT  $\rightarrow$  validación esquema  $\rightarrow$  persistencia.
- Propiedades:
  - Escalabilidad.
  - Seguridad.
  - Bajo acoplamiento.